

EduGenie

AI-Powered Educational Learning Assistant
Demo Mode Edition | FastAPI + HTML/CSS

PROJECT DOCUMENT

Table of Contents

1. Abstract
2. Introduction and Objectives
3. Use Case Scenarios
4. Key Features
5. System Architecture
6. Technology Stack and Pre-requisites
7. Project Structure
8. Project Workflow (Milestones)
9. Output Screenshots and Testing
10. Challenges Faced
11. Future Scope
12. Conclusion

1. Abstract

EduGenie is a lightweight, web-based educational learning assistant that uses generative-AI concepts to make studying simpler and more interactive. A student types a topic, a question, or a passage into a single text box and chooses one of four actions: **Ask Question**, **Generate Quiz**, **Learning Path** or **Summarize**. The result is displayed instantly in a response panel below the input.

The application is built with a **FastAPI** backend and a plain **HTML + CSS** frontend. This document describes the **Demo Mode** build, which runs completely offline on a local machine and **requires no Gemini API key**, so it can be demonstrated and tested by anyone. The modular design also allows the demo response engine to be replaced with Google Gemini for live, fully generative answers.

2. Introduction and Objectives

Many learners, especially beginners, find textbooks and search results overwhelming. They need short answers, quick self-assessment, a clear study plan and concise summaries in one place. EduGenie brings these learning tools together in a single, simple interface that works in any browser.

Objectives

- Provide short, accurate answers to general knowledge and academic questions.
- Test a learner's understanding by generating multiple-choice quizzes with an answer key.
- Provide structured, week-by-week learning paths from beginner to advanced level.
- Condense long educational passages into short bullet-point summaries for quick revision.
- Keep the system lightweight, modular and easy to run on ordinary hardware.
- Offer a Demo Mode so the project works without any paid service or API key.

3. Use Case Scenarios

Scenario	Description	Action used
1. General knowledge	A student studying geography asks "what is the largest ocean?" and instantly receives a short, direct answer.	Ask Question
2. Self-assessment	A student wants to check her understanding of the Pythagoras theorem and generates a five-question quiz with an answer key.	Generate Quiz
3. Skill roadmap	A learner exploring SQL requests a structured four-week plan with weekly topics and a daily practice target.	Learning Path
4. Quick revision	A student pastes a long passage on the importance of trees and gets a short bullet-point summary.	Summarize

4. Key Features

Feature	What it does
Ask Question	Returns a concise answer to a factual or academic question.
Generate Quiz	Creates multiple-choice questions (four options each) on a topic and provides an answer key.
Learning Path	Builds a time-boxed study plan with weekly topics and daily practice guidance.

Feature	What it does
Summarize	Turns a long paragraph into short, readable bullet points while keeping the core ideas.
Demo Mode	Runs without a Gemini API key; ideal for demonstrations, testing and offline use.
Simple UI	One text area, four buttons and a response panel – no learning curve.

5. System Architecture

EduGenie follows a simple client–server architecture. The browser sends the user's text to a FastAPI endpoint; each endpoint is connected to a dedicated module. In Demo Mode the module answers from the built-in response engine; in live mode the same module can call the Google Gemini API instead.

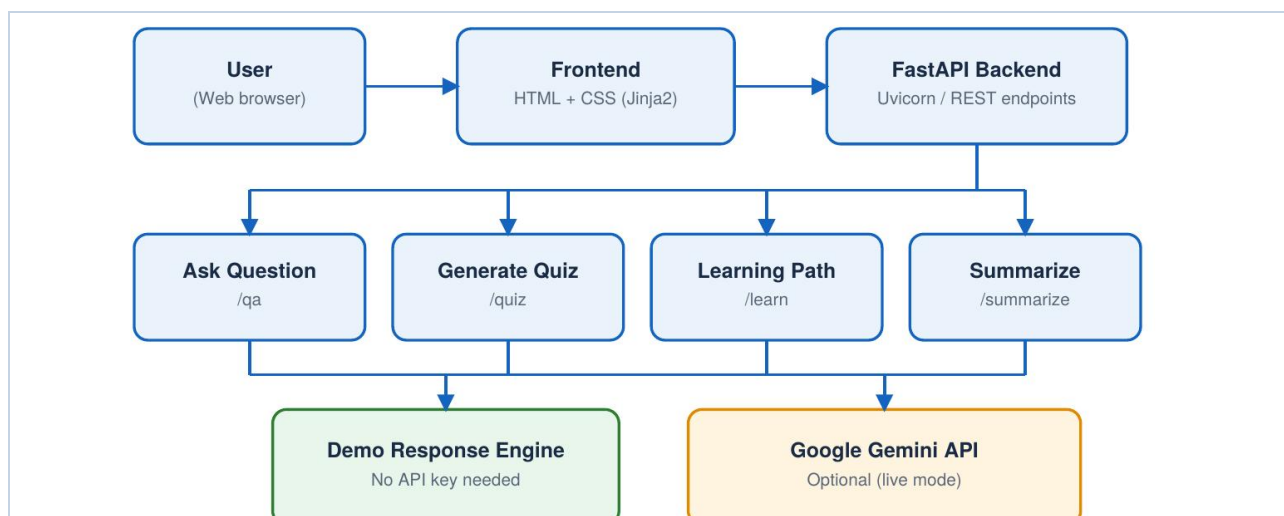


Fig. 1 – Architecture of EduGenie

Request flow

- The user enters a topic or passage in the text area and clicks one of the four buttons.
- The frontend sends the text to the matching FastAPI endpoint (question, quiz, learning path or summary).
- The corresponding module processes the request and produces the response text.
- FastAPI returns the result and the page displays it in the Response panel without a full reload.

6. Technology Stack and Pre-requisites

Component	Technology	Purpose
Language	Python 3.10+	Core application logic
Backend framework	FastAPI	REST endpoints and request handling
Server	Uvicorn (ASGI)	Runs the FastAPI application locally
Templating	Jinja2	Renders the HTML page
Frontend	HTML5 + CSS3	User interface and styling
AI service (optional)	Google Gemini API	Live generative responses (not needed in Demo Mode)

Component	Technology	Purpose
Tools	VS Code, Chrome, Git/GitHub	Development, testing and version control

Installation steps

- Install Python 3.10 or later and tick **Add Python to PATH** during setup; verify with `python --version`.
- Install dependencies: `pip install fastapi uvicorn jinja2` (or `pip install -r requirements.txt`).
- Optional (live mode only): create an API key in Google AI Studio and store it securely as an environment variable.

7. Project Structure

The project is organised into small, single-purpose files so each feature can be developed, tested and upgraded independently.

```
EduGenie/
|-- main.py                # FastAPI app and endpoints
|-- explanation_module.py  # Concept explanation logic
|-- qna.py                 # Question answering
|-- quiz_module.py         # Quiz generation
|-- summary_module.py      # Summarization
|-- learning_path.py       # Learning recommendations
|-- templates/index.html   # HTML frontend
|-- static/style.css       # Styling
|-- requirements.txt       # Python dependencies
```

Fig. 2 – Folder architecture

8. Project Workflow (Milestones)

Milestone 1 – Planning and Architecture

- Defined the four core features and the single-page user flow.
- Selected FastAPI + HTML/CSS for a lightweight, easy-to-run stack.
- Designed the modular folder structure with one module per feature.

Milestone 2 – Core Functionality

- Implemented the question-answer, quiz, learning-path and summary modules.
- Added a Demo Mode so every module returns a useful response with no API key.
- Exposed each module through a FastAPI endpoint.

Milestone 3 – Frontend Development

- Built index.html with a text area, four action buttons and a Response panel.
- Styled the page with style.css: centred card layout, blue theme, rounded buttons, monospaced response box.
- Connected the buttons to the backend so results appear directly below the input.

Milestone 4 – Deployment and Testing

- Run locally with `uvicorn main:app --reload`.
- Open `http://127.0.0.1:8000` in a browser.
- Functionally tested all four features (see Section 9).

9. Output Screenshots and Testing

The application was run locally at `127.0.0.1:8000` and each feature was tested with a sample input. The screenshots below show the actual output.

9.1 Ask Question

Input: *what is the largest ocean?* **Result:** the app answers that the Pacific Ocean is the largest ocean on Earth.

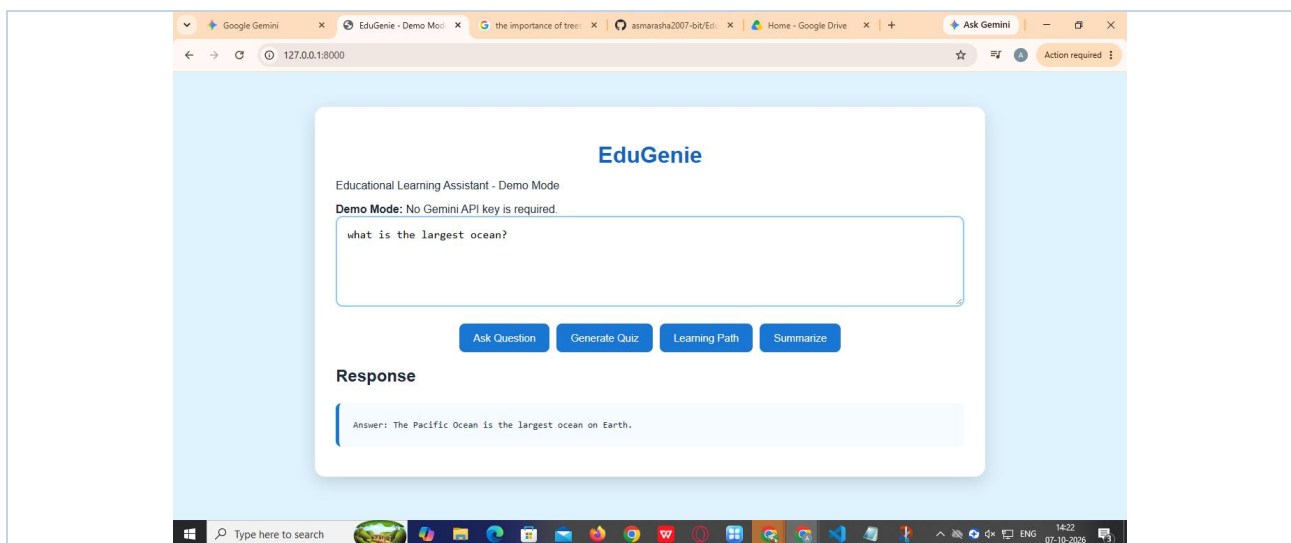


Fig. 3 – Ask Question: answer to “what is the largest ocean?”

9.2 Summarize

Input: a long paragraph on the importance of trees. **Result:** a short bullet-point summary that keeps the main ideas (air purification, oxygen release, soil protection).

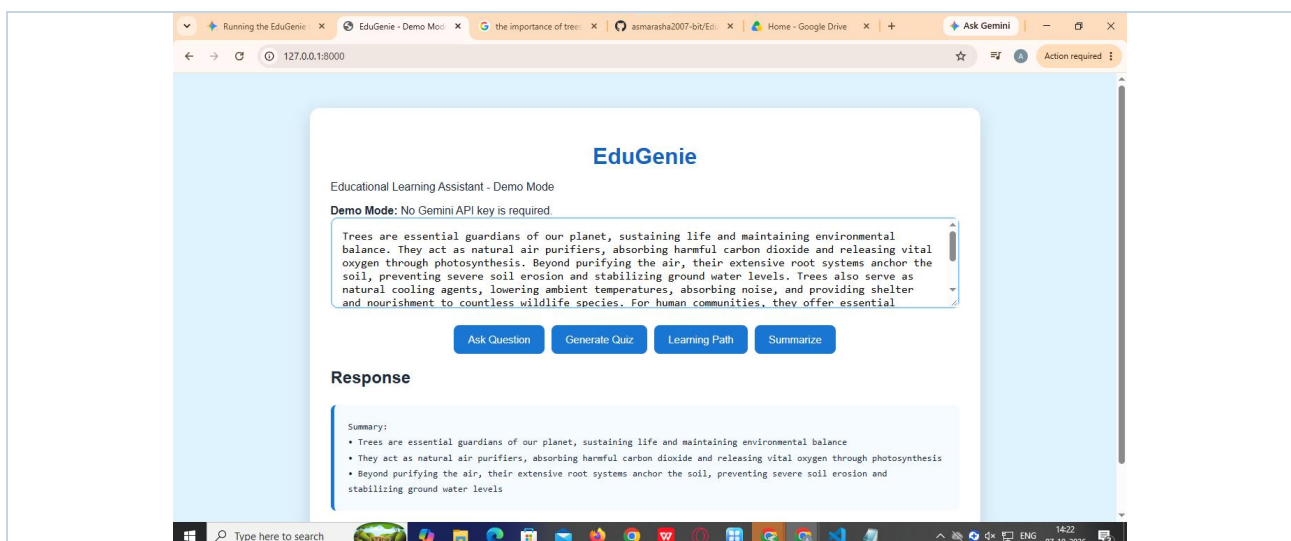


Fig. 4 – Summarize: bullet-point summary of a passage about trees

9.3 Generate Quiz

Input: *Pythagoras theorem* **Result:** a five-question multiple-choice quiz (formula, hypotenuse, a worked example, triangle type, squares) with an answer key: 1-B, 2-B, 3-A, 4-C, 5-D. The answers were checked and are correct.

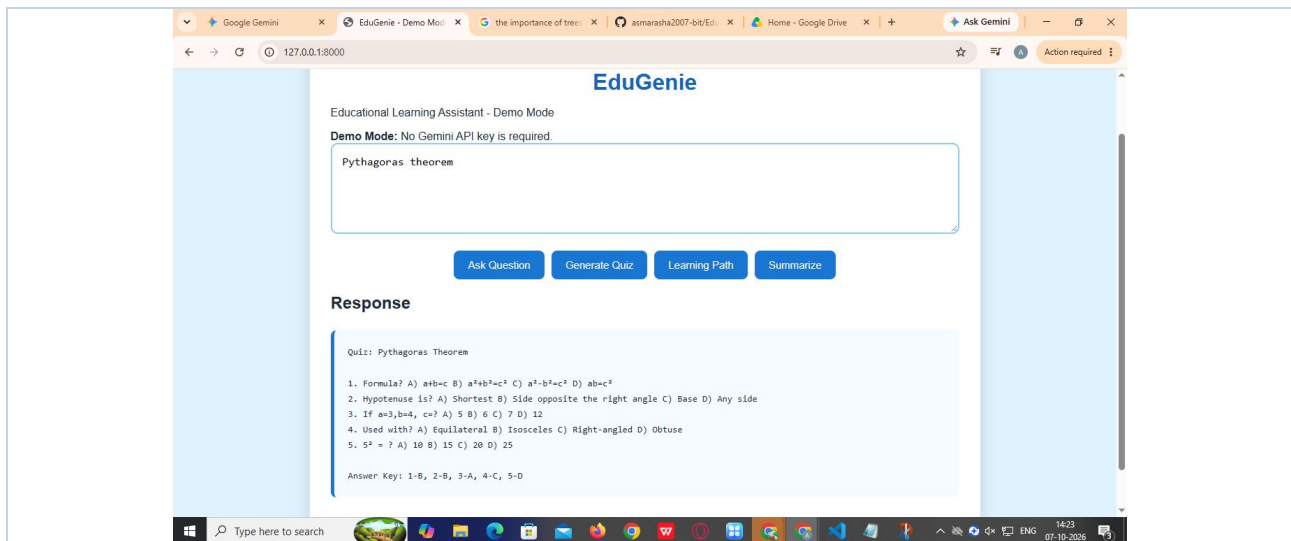


Fig. 5 – Generate Quiz: five MCQs with answer key for the Pythagoras theorem

9.4 Learning Path

Input: *sql* **Result:** a four-week SQL plan: basics and SELECT queries (Week 1), data modification and JOINS (Week 2), views, indexes and transactions (Week 3), and a student database project (Week 4), with a daily target of 45–60 minutes and at least five SQL queries.

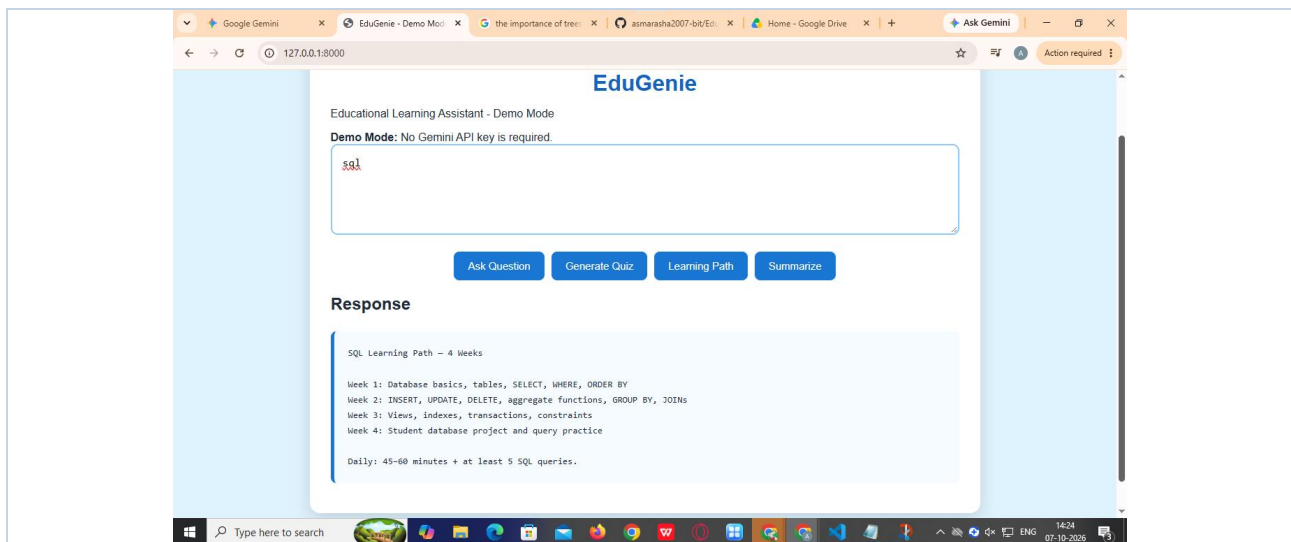


Fig. 6 – Learning Path: four-week SQL study plan

9.5 Test Summary

No.	Feature	Test input	Expected	Result
1	Ask Question	what is the largest ocean?	Pacific Ocean	Pass
2	Summarize	Passage on importance of trees	Short bullet summary	Pass
3	Generate Quiz	Pythagoras theorem	MCQs + correct answer key	Pass

No.	Feature	Test input	Expected	Result
4	Learning Path	sql	Week-wise plan with daily practice	Pass

10. Challenges Faced

- **No API dependency:** making the project demonstrable without a Gemini key required a built-in demo response engine.
- **Response accuracy:** quiz answers and plans had to be verified manually to make sure they were correct and useful.
- **Readable output:** formatting responses (bullets, numbered questions, weekly plans) so they are easy to read in a plain text panel.
- **Environment setup:** installing the right Python version and packages and running the server correctly on Windows.

11. Future Scope

- Switch from Demo Mode to live Google Gemini responses for any topic.
- Interactive quizzes with scoring, instant feedback and explanations.
- Voice input and multilingual support to reach a wider audience.
- User accounts with progress tracking, streaks and badges.
- Upload and summarise PDF or image notes.
- Mobile application and integration with learning platforms such as Google Classroom or Moodle.

12. Conclusion

EduGenie shows how a small, well-structured FastAPI application can deliver useful learning support: instant answers, quizzes, study plans and summaries from one clean interface. The Demo Mode build works entirely on a local machine with no API key, and all four features were tested successfully with the results shown in this document. Because the project is modular, it can grow from a demo into a fully AI-powered learning companion by connecting a generative model, adding progress tracking and supporting more languages and input types.